

Student Guide

Setting Up Databricks Mosaic AI in VS Code

CS5305

***Author:** Amna Iftikhar – TA (CS5305/CS621)*

Spring 2026

Contents

1.	Environment Prerequisites	3
1.1	Verify Python Installation	3
2.	VS Code Setup.....	4
2.1	Required Extensions	4
2.2	Create a New Databricks Project.....	4
2.3	Enter Workspace URL.....	5
2.4	Select Authentication Method	5
2.5	Name Your Profile and Authenticate	5
3.	Databricks Connect Setup.....	7
3.1	Create Configuration	7
3.2	Select a Cluster	7
3.3	Set Up Python Virtual Environment	7
3.4	Install Databricks Connect.....	8
3.5	Verify Setup is Complete	8
4.	Mosaic AI & Repository Setup	9
4.1	Fork the Course Repository	9
4.2	How to Fork	9
4.3	Clone Your Fork	10
4.4	Generate Your Databricks Token	10

1.Environment Prerequisites

Before starting, make sure you have the following ready:

- **VS Code** — Latest version installed on your PC
- **Python Version 3.12** (You could have other versions installed as well)
- **Microsoft or Google account** — Used to log in to Databricks Account

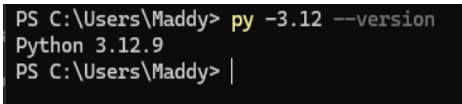
Note: Python 3.12 is specifically required. Databricks extensions will NOT work with any other python version.

1.1 Verify Python Installation

Before moving on, open a new cmd/shell windows and run the following command to verify the required version of python.

```
py -3.12 --version
```

You should see Python 3.12.9 confirming successful installation.



```
PS C:\Users\Maddy> py -3.12 --version
Python 3.12.9
PS C:\Users\Maddy> |
```

Figure 2: Terminal confirming Python 3.12.9 is installed

If you don't have the required version of python installed, go to <https://www.python.org> and follow the instructions to install Python 3.12.9 on your machine.

2.VS Code Setup

This section covers installing the required VS Code extensions and creating your Databricks project inside VS Code.

2.1 Required Extensions

Ensure you the following extensions installed in VS Code:

1. **Python** — by Microsoft
2. **Jupyter** — by Microsoft
3. **Databricks** — by Databricks (red logo — verified publisher)

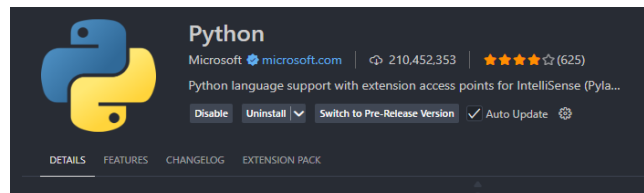


Figure 3: Python Extension by Microsoft — installed

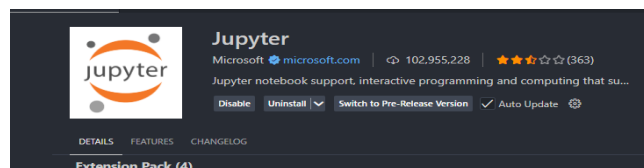


Figure 4: Jupyter Extension by Microsoft — installed

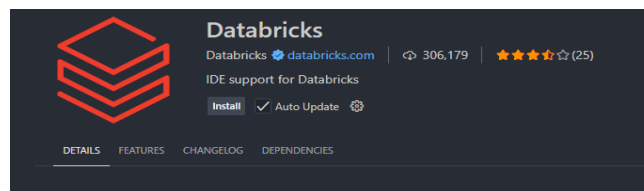


Figure 5: Databricks Extension (red logo) — click Install

Tip: Make sure the Databricks extension shows "databricks.com" as the verified publisher. Do not install any other unofficial alternatives.

2.2 Create a New Databricks Project

Create an empty folder on your Desktop named **databricks-project**. Open VS Code, press **Ctrl+K** then **Ctrl+O** and select that folder. Then click the **Databricks icon** (red logo) in the left sidebar and click **"Create a new Databricks project"**.

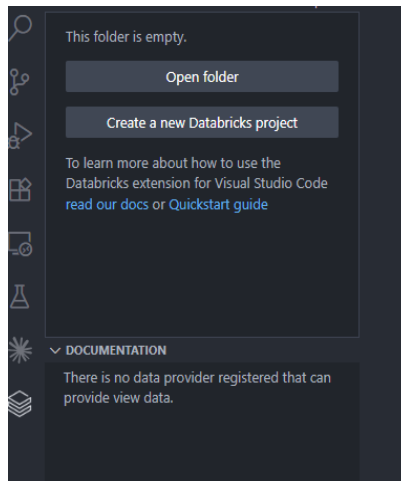


Figure 6: Click "Create a new Databricks project"

2.3 Enter Workspace URL

A prompt will appear asking for the Databricks Host. Enter the URL for your databricks workspace and press Enter:

```
https://<your_workspace_url>.cloud.databricks.com
```

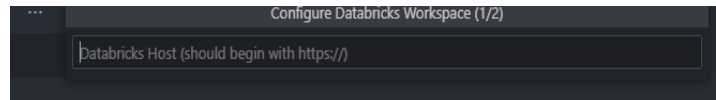


Figure 7: Enter the Databricks workspace URL

2.4 Select Authentication Method

Select **"OAuth (recommended)"** — this opens a browser to log you in.

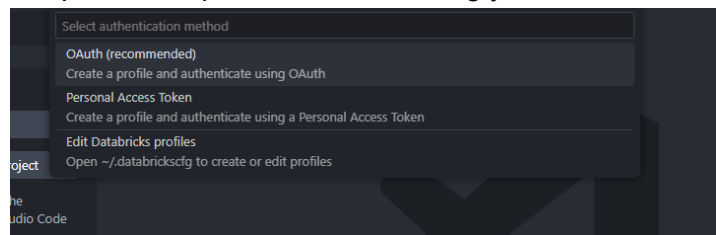


Figure 8: Select OAuth (recommended)

2.5 Name Your Profile and Authenticate

When prompted for a profile name, type **DEFAULT** and press Enter.

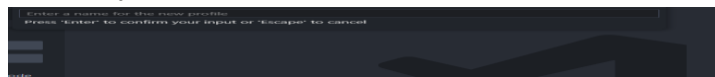


Figure 9: Type DEFAULT as the profile name

A browser will open — sign in with your **Microsoft or Google account** that you used to signup for Databricks. Once you see "Authenticated", close the browser and return to VS Code.

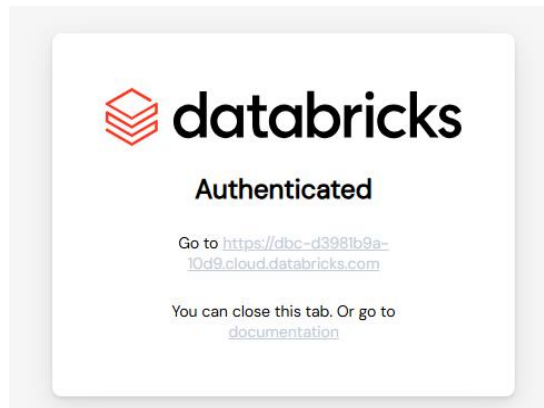


Figure 10: Databricks Authentication Successful

3.Databricks Connect Setup

This section covers connecting VS Code to Databricks, setting up the Python environment, and installing Databricks Connect automatically.

3.1 Create Configuration

After authenticating, you will see the **CONFIGURATION** panel on the left. Click **"Create configuration"** and when the workspace URL appears, click on it to select it.

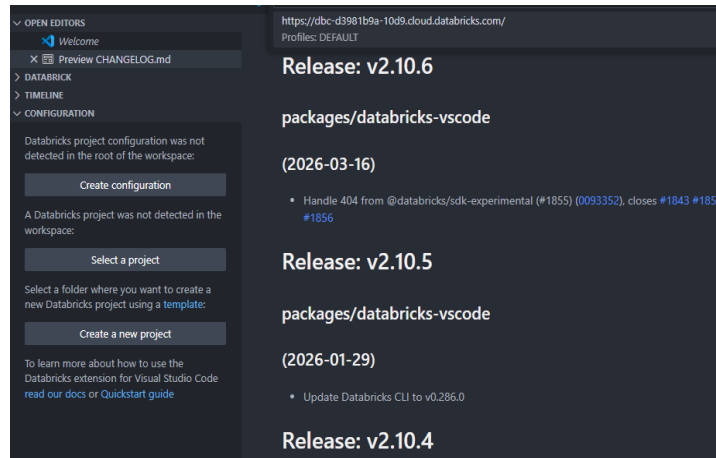


Figure 11: CONFIGURATION panel — click "Create configuration"

3.2 Select a Cluster

A dropdown will appear. Select **"Serverless"** which will allow you to run notebooks without a dedicated cluster.

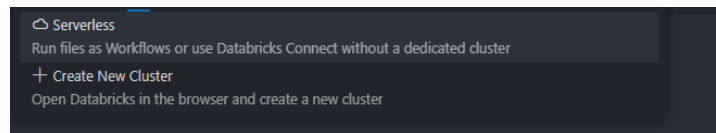


Figure 12: Select Serverless as the compute resource

3.3 Set Up Python Virtual Environment

In the CONFIGURATION panel under **Python Environment**, click **"Activate an environment with Python..."**.

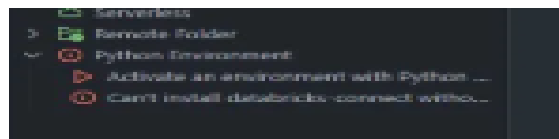


Figure 13: Click "Activate an environment with Python..."

Select **"Venv"** — the extension will create the virtual environment automatically.

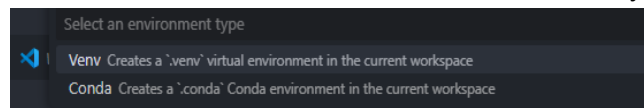


Figure 14: Select Venv to create the environment automatically

Then select **"Python 3.12.9"** from the list.

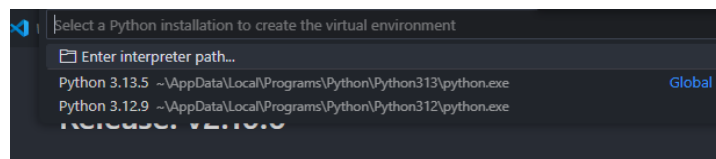


Figure 15: Select Python 3.12.9 from the list

3.4 Install Databricks Connect

A popup will appear at the bottom right. Click the **"Install"** button. The extension will automatically download and install all required packages — no manual pip commands needed.

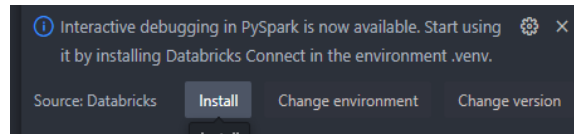


Figure 16: Click "Install" to automatically install Databricks Connect

The OUTPUT panel will show all packages being installed. Wait for this to complete — do not close VS Code.

Note: Do not close VS Code during installation. Wait until you see "Databricks Connect enabled" at the bottom status bar.

```
Collecting platformdirs<=2.5 (from jupyter-core!=5.0.*,>=4.12->nbformat)
  Downloading platformdirs-4.9.4-py3-none-any.whl.metadata (4.7 kB)
Requirement already satisfied: typing-extensions<=4.4.0 in c:\Users\Maddy\Desktop\databrick\.venv
  (from referencing<=0.28.4->jsonschema<=2.6->nbformat) (4.15.0)
Downloading nbformat-5.10.4-py3-none-any.whl (78 kB)
Downloading fastjsonschema-2.21.2-py3-none-any.whl (24 kB)
Downloading jsonschema-4.26.0-py3-none-any.whl (90 kB)
Downloading attrs-26.1.0-py3-none-any.whl (67 kB)
Downloading jsonschema-specifications-2025.9.1-py3-none-any.whl (18 kB)
Downloading jupyter_core-5.9.1-py3-none-any.whl (29 kB)
Downloading platformdirs-4.9.4-py3-none-any.whl (21 kB)
Downloading referencing-0.37.0-py3-none-any.whl (26 kB)
Downloading rpsd_py-0.30.0-cp312-cp312-win_amd64.whl (240 kB)
Downloading traitlets-5.14.3-py3-none-any.whl (85 kB)
Installing collected packages: fastjsonschema, traitlets, rpsd-py, platformdirs, attrs, referenc
jsonschema-specifications, jsonschema, nbformat

Successfully installed attrs-26.1.0 fastjsonschema-2.21.2 jsonschema-4.26.0 jsonschema-specificat
jupyter-core-5.9.1 nbformat-5.10.4 platformdirs-4.9.4 referencing-0.37.0 rpsd-py-0.30.0 traitlets
Databricks Connect enabled 0 0 0
```

Figure 17: Packages being installed automatically — wait for completion

3.5 Verify Setup is Complete

Once complete, the CONFIGURATION panel will show green checkmarks for all items:

- **Local Folder** — your project folder
- **Target** — dev
- **Auth Type** — OAuth profile
- **Serverless** — selected
- **Active Environment:** .venv — created automatically
- **Databricks Connect: 15.1.0** — installed automatically

The bottom status bar will show **"Databricks Connect enabled"** confirming everything is working.

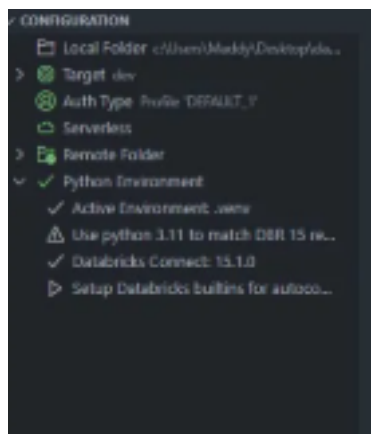


Figure 18: Final CONFIGURATION panel — Databricks Connect enabled

4. Mosaic AI & Repository Setup

This section covers forking the course repository and converting the Azure OpenAI notebooks to run on Databricks Mosaic AI.

4.1 Fork the Course Repository

You should fork the repository instead of cloning it directly for the following reasons:

- **Your own copy:** Forking creates your own copy of the repo under your GitHub account. You can make changes, add notes, and modify notebooks without affecting the original.
- **Avoid overwriting:** If the instructor updates the original repo, a direct clone could cause conflicts. A fork keeps your work separate and safe.
- **Pull updates safely:** You can sync your fork with the original repo to get new content without losing your own changes.

Note: Always fork the original repository first, then clone your own forked repository — not the original repo directly.

4.2 How to Fork

4. Open <https://github.com/alikhawaja/cs5305> in your browser

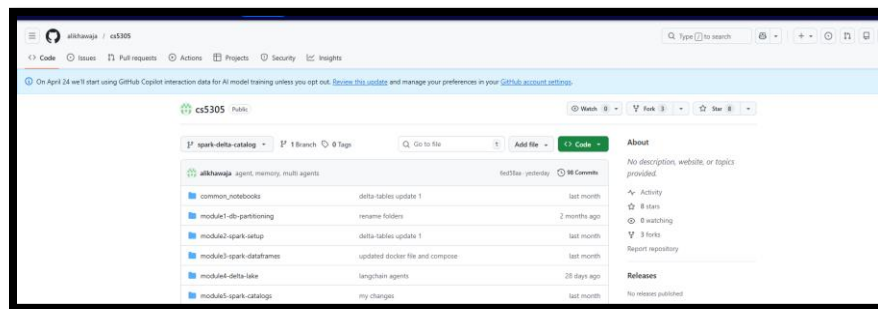


Figure 19: Original course repo — click the Fork button (top right)

5. Click the **Fork** button in the top right corner of the page
6. Select your own GitHub account as the destination

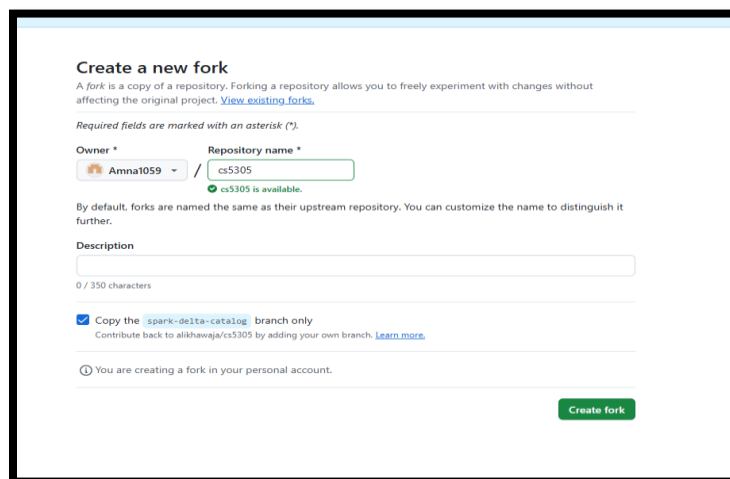


Figure 20: Create a new fork — keep default name and click Create fork

7. You can keep the repository name as **cs5305**, or provide your own name (e.g., langchain_training) and click **Create fork**
8. Wait for GitHub to create your fork — you will be redirected to your own copy

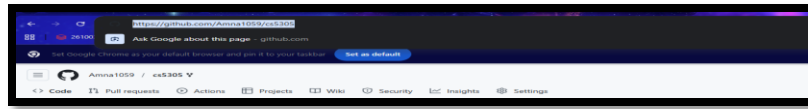


Figure 21: Your forked repo URL — notice it shows YOUR username/cs5305

4.3 Clone Your Fork

After forking, clone YOUR fork (not the original) by running in the VS Code terminal:

```
git clone https://github.com/<your_github_account_name>/<your_repo_name>
cd <your_repo_name>
```

4.4 Generate Your Databricks Token

To run notebooks locally using Mosaic AI, you need a personal access token from Databricks:

9. Open https://<your_databricks_workspace>.cloud.databricks.com in your browser

10. Click your **profile icon** (top right) → **Settings**

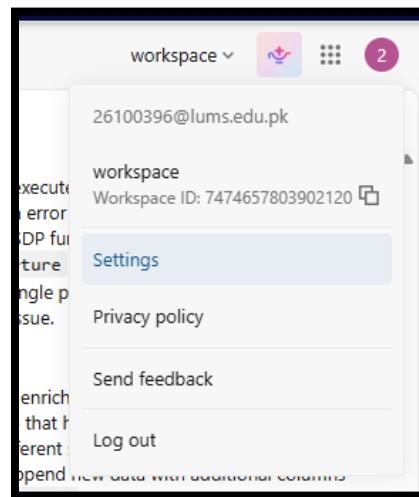


Figure 22: Go to Settings from your profile icon

11. Go to **Developer** → **Access Tokens**

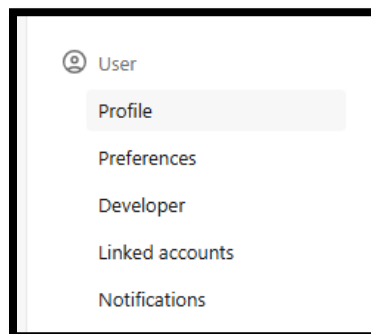


Figure 23: Click Developer → Access Tokens

12. Click **Generate New Token**

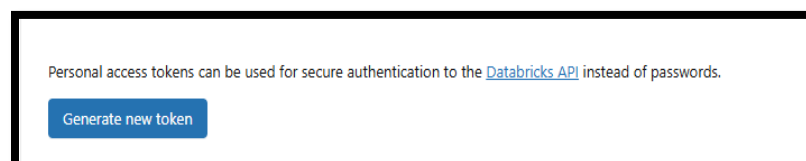


Figure 24: Click Generate New Token button

13. Give it a name like "vscode-test", set 90 days lifetime, select "all APIs" in scope

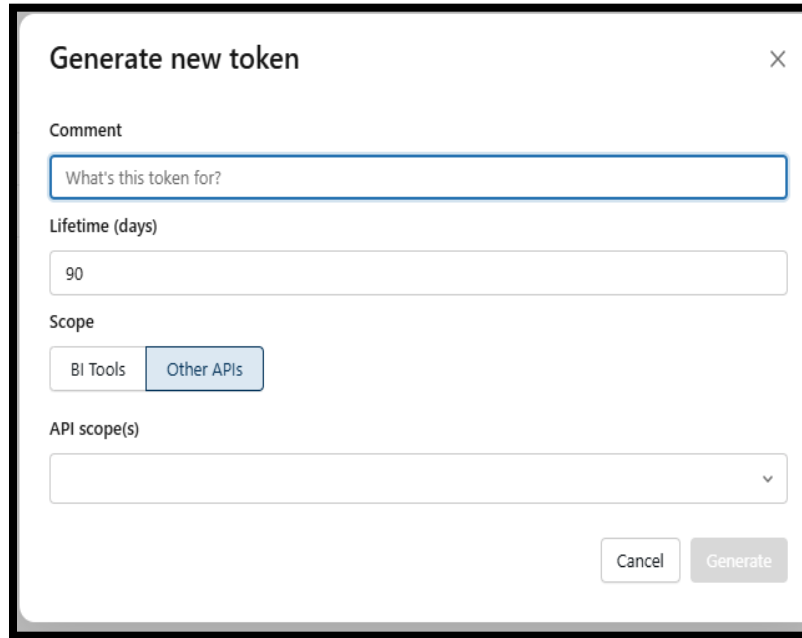


Figure 25: Fill in token details and click Generate

14. **Copy the token immediately** — it will NOT be shown again

Note: Keep your token private. Never share it or commit it to GitHub. If exposed, delete it and generate a new one immediately.

4.3 Create a .env File for Your Credentials

Instead of hardcoding your token inside every notebook, the correct professional approach is to store all credentials in a single .env file.

Why Use a .env File?

- **Security:** Your token stays in one place and never appears inside notebook code
- **Prevent accidental leaks:** If you commit notebooks to GitHub, your token will NOT be exposed
- **Easy to update:** If your token expires, update only one file — not every notebook
- **Industry standard:** This is how real projects manage credentials professionally

Note: Never commit your .env file to GitHub. It is already added to .gitignore in the repo — do not remove it.

Step 1 — Create the .env File

In the root of your cloned cs5305 folder, create a new file named **.env**. In VS Code Explorer, right-click the cs5305 folder → **New File** → name it **.env**

Step 2 — Add Your Credentials

Open the .env file and add these two lines:

```
DATABRICKS_HOST=https://<your_databricks_workspace>.cloud.databricks.com
DATABRICKS_TOKEN=your_token_here
```

Replace your_token_here with the actual token you copied in Step 4.2.

Tip: Do not add quotes around the values in the .env file. Write them exactly as shown above.

Step 3 — Install python-dotenv

Run this in the VS Code terminal to install the package that reads .env files:

```
pip install python-dotenv
```

4.4 Convert Azure OpenAI Notebooks to Mosaic AI

The course notebooks use Azure OpenAI at 3 levels. Each level needs a specific conversion. Follow all steps in order.

Step 1 — Load Credentials from .env

At the very top of every notebook, replace the Azure environment variable check with this:

```
REMOVE (Azure):  
import os  
from dotenv import load_dotenv  
load_dotenv()  
# checks for AZURE_OPENAI_API_KEY, AZURE_OPENAI_ENDPOINT etc.
```

REPLACE with (Mosaic AI):

```
import os  
from dotenv import load_dotenv  
load_dotenv() # loads DATABRICKS_HOST and DATABRICKS_TOKEN from .env  
# Verify your variables loaded correctly:  
print("DATABRICKS_HOST:", "SET" if os.getenv("DATABRICKS_HOST") else "NOT SET")  
print("DATABRICKS_TOKEN:", "SET" if os.getenv("DATABRICKS_TOKEN") else "NOT SET")
```

Note: Make sure your .env file is in the root of the cs5305 folder with DATABRICKS_HOST and DATABRICKS_TOKEN set.

Step 2 — Level 1: Replace Native API Client

The first level uses the Azure OpenAI native client directly. Replace it with the Databricks SDK client:

REMOVE (Azure Native API):

```
from openai import AzureOpenAI  
client = AzureOpenAI(api_version="2023-03-15-preview")  
response = client.chat.completions.create(  
    model="lums-gpt-4.1-mini",  
    temperature=0,  
    messages=[{"role": "user", "content": "your prompt"}]  
)  
result = response.choices[0].message.content
```

REPLACE with (Mosaic AI Native):

```
from databricks.sdk import WorkspaceClient  
from databricks.sdk.service.serving import ChatMessage, ChatMessageRole  
  
client = WorkspaceClient()  
response = client.serving_endpoints.query(  
    name="databricks-meta-llama-3-3-70b-instruct",  
    messages=[ChatMessage(role=ChatMessageRole.USER, content="your prompt")]  
)  
result = response.choices[0].message.content
```

Step 3 — Level 2: Replace LangChain LLM Object

The second level creates a LangChain LLM object. This is the most important change — you are replacing the Azure model instance with a Mosaic AI model instance:

REMOVE (Azure LangChain):

```
from langchain_openai import AzureChatOpenAI
llm = AzureChatOpenAI(model="lums-gpt-4.1-mini")
```

REPLACE with (Mosaic AI LangChain):

```
from databricks_langchain import ChatDatabricks

llm = ChatDatabricks(
    endpoint="databricks-meta-llama-3-3-70b-instruct",
    temperature=0,
    max_tokens=500
)
```

Tip: Once you replace the llm object, all the code below that uses llm stays exactly the same — invoke(), chains, everything.

You can immediately test your new LLM object just like the original notebook does:

```
msg = llm.invoke("What is Python in one sentence?")
print(msg.content)
```

Step 4 — Level 3: LCEL Chains Stay the Same

Good news — if you replaced the llm object correctly in Step 3, all LCEL chains work without any changes:

```
NO CHANGES NEEDED:
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in 3 short bullet points."
)

chain = prompt | llm | StrOutputParser() # same as before
result = chain.invoke({"topic": "LangChain"})
print(result)
```

Note: The chain uses the pipe operator (|) which works with any LangChain-compatible LLM — including ChatDatabricks. No changes needed here.

Replace Embeddings (if used)

REMOVE (Azure):

```
from langchain_openai import AzureOpenAIEmbeddings
embeddings = AzureOpenAIEmbeddings(...)
```

REPLACE (Mosaic AI):

```
from databricks_langchain import DatabricksEmbeddings
embeddings = DatabricksEmbeddings(endpoint="databricks-bge-large-en")
```

Select the Correct Kernel

When opening any notebook, click the kernel selector (top right) and select **.venv (Python 3.12.9)**

Available Mosaic AI Models

- **General chat:** "databricks-meta-llama-3-3-70b-instruct"
- **Fast/lightweight:** "databricks-meta-llama-3-1-8b-instruct"
- **Embeddings:** "databricks-bge-large-en"

Tip: You are now fully set up! 1) Fork the repo, 2) clone your fork, 3) create your .env file, open any notebook, 4) apply the 4 conversion steps, 5) select the .venv kernel, and run — your code will execute on Databricks Mosaic AI directly!